AgentForge 项目白皮书

2026 第五届「未来杯」创新大赛 · 赛题一 Agent 编排创新应用

项	内容
项目名称	AgentForge（可验证的多智能体开发编排工作台）
参赛队伍	奶龙队
学校	湖北工业大学工程技术学院
成员	王拙诚、吴捍卫、黄浩
指导教师	刘浪
队伍编号	2117173210695860225
提交命名	赛题一+奶龙队+AgentForge
版本	v1.0（2026-11-12）

> 配套材料：项目方案书（技术详解）、方案讲解 PPT、演示视频、公开源码仓库与证据索引。本白皮书按官方「项目白皮书」五部分组织：项目概述、解决方案、应用成效、商业模式、团队介绍。

一、项目概述

1.1 项目背景

大模型应用正从单轮对话走向多智能体协作。赛题指出当前 Agent 工程化的三大痛点：编排能力薄弱（多为脚本化、硬编码流水线，缺乏动态分支、循环与异常回退）、调度策略单一（串行或广播式通信，缺乏基于任务优先级、资源负载与能力画像的智能调度）、验证与可信机制缺失（中间结果缺乏自动验证、交叉核对与人工介入节点，幻觉与错误传播难以收敛）。赛题要求构建覆盖「Agent 定义与配置、工作流编排、任务调度与执行、结果验证与监控、反馈循环与优化」五大模块，并至少包含一种自我循环机制的**可运行**多智能体平台。

1.2 应用范围

AgentForge 是面向「大模型 Agent 工具编排调度」的本地工作台，第一套行业场景是小型软件项目的需求实现与持续改进；平台主体是通用编排器，替换 Agent、工具与工作流即可迁移到其他场景：

- **高校课程与实训**：学生可观察任务图、调度依据、Docker 验收回执与失败原因，把「AI 如何分工、为何失败」变成可讲解、可复盘的实验材料；本地部署满足机房网络与数据要求。
- **科研小工具**：数据脚本、CLI、小型 Web 页等边界清晰、验收可自动化的任务，可快速实现并附带可复现证据包。
- **企业研发提效**：模型与工具可替换、源码不出内网、候选只在受控容器执行、验收可审计，适合内部工具与低风险功能开发。

当前边界如实声明：只接受有界标准库 Python 多文件项目、JSON 可传递接口，以及固定 wheel 依赖的 Python Web 项目；不支持任意大型仓库、自动安装任意依赖或常驻生产服务；未部署公网、无多用户认证。

1.3 核心优势

1. **模型驱动的动态编排，而非硬编码流水线**：主 Agent 依据需求、上一批真实结果与验收反馈决定派发、补派、取消、验证或收尾；宿主只强制模型不能越过的约束（权限、依赖、写入互斥、并发、预算与完成判据）。
2. **独立验收 + 哈希绑定，结果可验证**：候选代码只在隔离 Docker 中执行冻结验收，候选哈希、容器回执 nonce 与独立语义审查三者绑定；模型自报「通过」不改变任何结论。
3. **事件、快照与复现包，运行可审计可复现**：调度依据、工具回执、失败分类、修正过程全部落事件并可在协作图与时间轴回放；每批运行导出带 SHA256 清单的复现包。

二、解决方案

2.1 架构设计

```
[交互层] React 19 + TypeScript 7 + Vite 8：对话、项目、协作图、交付页
    | HTTP /api（40 条路由，单端口）
[API 层] FastAPI + SQLite（5 库 16 张表）：运行/事件/步骤/快照/版本/验收回执
    |
[编排内核] 模板 DAG | 模型驱动动态编排 | 项目与 Web 模式
    | 能力+权限+依赖+优先级+负载调度
[执行与工具层] 模型适配、知识检索、文件编辑、项目/Web 工具
    | 候选源码
[隔离验收层] WSL2 内 Docker Engine：禁网、只读挂载、非 root、capabilities 丢弃、资源限额
    | 验收回执 + 候选哈希
[证据层] 事件、快照、独立审查、ZIP 复现包（SHA256 清单）
```

规模：后端 21 个模块 6,958 行、40 条 API；前端 19 个文件 3,268 行；测试 53 个后端测试文件。模型接入为 OpenAI 兼容 `POST /chat/completions`（当前 `deepseek-v4.1-flash`），密钥只存服务端环境变量。

2.2 方案功能（对应赛题五模块）

赛题要求	AgentForge 实现
Agent 定义与配置	定义名称、角色、能力标签（plan/develop/test/review/fix）、提示词、模型与工具白名单；每次保存生成新版本，历史运行固定当时配置与知识版本
工作流编排	代码化模板 DAG（顺序、并行、条件、有限循环、子工作流）+ 运行中由模型生成的动态任务图；协作图可视化回放
任务调度与执行	先按能力与工具权限筛选，再结合依赖、优先级、当前负载与在途 token 预留选择执行者；动态编排按依赖成功与写入路径互斥并行（≤3）；调度依据、候选列表与失败重派全部留事件
结果验证与监控	冻结验收在独立 Docker 执行；可信驱动在父进程、候选由 worker 子进程经受限 JSON 协议提供行为；成功需同时满足测试通过、隔离成立、退出码为零、审查 `passed`、候选哈希一致；实时轮询状态与失败分类
反馈循环与优化	真实失败反馈给修正 Agent，模板路径最多两轮；模型编排路径由主 Agent 创建修复任务或替换失败任务；`finishing-budget-v1` 在原有总预算内为最终验收与审查预留资源

自我循环机制：执行 → 独立验收 → 结构化反馈 → 修正 → 复验 → 审查（≤2 轮），失败不静默重跑；模型可在失败后创建新 ID 的替代任务；预算接近上限时按门控收尾，不通过扩大预算或跳过检查获得「成功」。

知识库与记忆：`knowledge/*.md` 与项目资料按版本冻结，关键词检索返回片段与文件路径（≤3 条）进入模型上下文，事件记录 `knowledge_retrieved`；记忆为任务级（冻结上下文、事件、快照），不是跨任务长期学习。如实说明：未使用向量数据库，也未接入 MCP/A2A 标准协议。

2.3 技术内容

- **可信隔离**：候选 Python 只经 Docker 验收，从不宿主机执行；网络关闭、根文件系统只读、非 root（uid 65534）、capabilities 丢弃、`no-new-privileges`、内存/CPU/进程限额、外层超时清理；Web 预览使用独立 origin + capability cookie + CSP + iframe sandbox。
- **证据门控**：候选哈希、冻结验收哈希、容器回执与审查结论互锁；候选变化使旧验收立即失效；应用候选需用户显式确认并复核哈希与目标基线。
- **可追溯记录**：SQLite 追加事件与原子节点检查点（成功节点与恢复上下文同一事务提交）；服务重启把运行中状态改为阻塞并保留恢复入口。
- **实验与复现**：三组对照运行器（单 Agent/固定串行/动态并行）按预注册预算与终点执行，失败不补跑；每批导出带逐文件 SHA256 的复现包。

三、应用成效

3.1 总述

平台已完成标准库 Python 多模块库、CLI 持续迭代、Flask Web 开发与预览、四项固定功能任务批量评测等真实任务；所有「通过」以独立容器验收 + 审查为准，候选默认不应用。2026-11-12 本机复核：`preflight.py` 退出码 0（模型与 WSL Docker 可用）、Docker 沙箱探针 6/6 通过；公开仓库干净克隆后

后端 **872 passed + 278 subtests**、前端 **110 passed**、`npm ci` + 构建通过且产物哈希与交付记录一致。

3.2 案例一：Flask 工作项看板（Web 项目）

主 Agent 读取健康页项目后自行拆出后端 API 与前端资产两个无依赖任务并真实并行执行；首个自然候选通过 **14 项生成 HTTP 验收**与维护者独立冻结的 **12 项 HTTP 复核**（10 请求、131,918 tokens），随后在隔离 iframe 中完成创建、完成切换、筛选、空标题校验与 390px 布局。独立预设缺陷实验删除完成状态过滤后，真实 Docker 验收先失败（12 项中 1 项断言失败），模型修复后复验 **12/12** 通过（4 请求、72,644 tokens）。该修正来自预设缺陷，不计为自然自修复。

3.3 案例二：记账 CLI 持续开发

同一项目从 v1 导入（未验收）到 v2 分类报表 **13/13**、再到 v3 删除记录 **20/20**，两轮 18 请求、136,769 tokens；回归验收逐版本继承、旧版本不可变。第二轮主 Agent 自主选择 `edit → tool × 5 → verify → finish`，五次 Docker 应用工具结果真实进入后续决策，但应用退出 0 不授予验收通过。

3.4 案例三：四项任务与三组对照

- 四项固定任务（完成状态、原子批量编辑、优先级分页、JSON 合并）各 10 项独立验收，修复后各 **10/10**（43 请求、313,562 tokens）。
- 同模型、同任务、同预算的三组 24 次配对对照：功能验收单 Agent **6/8**、固定串行 **6/8**、动态并行 **8/8**；完整流程 6/8、6/8、7/8；合计 174 请求、已知 1,150,622 tokens，保留 4 次 HTTP 429、13 次格式重试与 1 次本地中断。样本小、任务同属一个代码库，只作描述性报告，不宣称普适提升。

3.5 复现与证据

9 个历史复现包与同名 `.sha256` 一一对应（2026-11-12 复核 9/9 通过），包含源码、冻结请求、候选、事件与快照；公开仓库 <https://github.com/yancheng0518/agentforge-future-cup>（tag `v1.2-submission-20261112`）提供干净克隆复现记录。**未完成**：换机从零安装（第二台机器）、并发/压力测试与自然失败自修正样本。

四、商业模式

4.1 推广模式

以「开源核心 + 课程实训包 + 企业私有化部署」三级推进：开源核心建立技术信任与社区；课程包提供案例、冻结验收库与实验指导书；企业版提供私有部署、模型适配、审计报告与技术支持。当前无公网部署与多用户认证，SaaS 形态暂缓。

4.2 市场空间

AI 编程与 Agent 平台快速增长但同质化明显。本项目差异点是「验证可信 + 调度可解释 + 本地可审计」，切入对合规与内网敏感的高校与中小企业研发场景。不作具体市场规模断言，也不宣称已有商业验证；与通用编程助手比的是「多 Agent 协作是否可验证、失败是否可解释、结果是否可复现」，而非代码补全速度。

4.3 生态运作

短期兼容主流模型 API 与 Docker 工具链；中期接入 MCP/A2A 复用外部工具生态；长期建设验收模板与课程案例市场。收入设想为课程授权与实训服务、企业私有化部署与支持、验收模板订阅，均需在决赛后试点验证。阶段目标：初赛后开展 2–3 个高校课程试点与 1 个企业内部试用，验证可用性、支持成本与续费意愿，再决定是否投入多租户与公网版本。

五、团队介绍

5.1 队伍与学校

奶龙队（项目负责人：王拙诚），湖北工业大学工程技术学院，队伍编号 2117173210695860225；成员王拙诚、吴捍卫、黄浩，指导教师刘浪。

5.2 成员分工

成员	建议分工
王拙诚	项目负责人；编排内核与后端（调度、验证门控、SQLite 证据链）
吴捍卫	沙箱与验证（Docker 隔离、验收协议、失败分类）
黄浩	前端与可视化（对话、协作图、项目交付页）、测试与材料

> 实际分工以团队内部确认为准；本表为依据项目模块的建议划分，不替代成员履历。

5.3 核心成员履历

【待补充：各成员年级/专业、相关课程或项目经历、竞赛获奖、负责模块与代码贡献】。

5.4 团队核心资质与优势

【待补充：学校/学院与实验室支持、指导教师研究方向、团队已有的 AI 或软件工程实践基础、可提供的试用或应用证明】。

六、相关证明材料清单

材料	状态
学生身份证明（学生证/在读证明/学信网学籍在线验证报告）	待上传（团队自备）
团队基本资质证明	待上传（如适用）
应用推广证明（用户/教师试用反馈）	暂无；未开展的不虚构
项目介绍 PPT、项目介绍视频、源码仓库与说明文档	已随初赛材料提交

七、附录：材料与证据索引

- 方案书（技术详解）：`submission/AgentForge-方案书.pdf`

- 方案讲解 PPT：`submission/项目介绍.pptx`
- 演示视频：`submission/演示视频.mp4`（4:02，SHA256 见 `演示视频.sha256`）
- 评审版说明：`submission/README-评审版.md`
- 证据索引：`submission/证据索引.md`（9 个复现包 SHA256、docs 分组、真实/预设/模拟分类）
- 干净克隆复现记录：`submission/复现验证记录-20261112.md`
- 源码仓库：<https://github.com/yancheng0518/agentforge-future-cup>（`main`，tag `v1.2-submission-20261112`）

> 数字均来自项目运行记录与文档；团队对内容真实性负责，未验证项已如实标注。